From Max-Cut to Semidefinite Programming, EVA, and Clustering-Based Rounding

A step-by-step introduction, from graphs and matrices to reproducible G-set experiments

Abstract

Max-Cut asks how to split the vertices of a weighted graph into two groups so that the total weight of the edges crossing between them is as large as possible. Searching over all splits is generally intractable. We build the problem from first principles, reformulate it using signs and matrices, and derive a *semidefinite programming* (SDP) relaxation. We then explain the geometry of unit vectors, how the nonconvex *Efficient Vector Ascent* (EVA) algorithm optimizes a low-rank factor of the SDP variable, and how three different rounding methods—Goemans–Williamson random hyperplanes, K-Means2N, and K-MeansNM—convert continuous vectors into an actual graph partition. We conclude with the G-set benchmark, convergence and precision, independent random restarts, and an exact description of the project's saved-factor workflow. All formulae concern *Max-Cut* ($k = 2$), with the distinction from general Max-k-Cut stated explicitly.

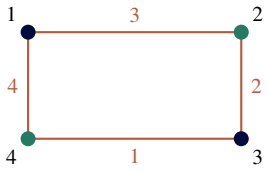


Figure 1. A weighted four-cycle. The blue vertices $\{1, 3\}$ and green vertices $\{2, 4\}$ form a cut of weight $3 + 2 + 1 + 4 = 10$. Every edge crosses.

1 What are we trying to solve?

Graphs. A *graph* is a set of vertices (objects) joined by edges (relationships). Write

$$G = (V, E, w), \qquad V = \{1, \dots, n\}. \tag{1}$$

Here E contains pairs $\{i, j\}$ of vertices; $w_{ij} = w_{ji} \in \mathbb{R}$ is the weight of their edge, and we use $w_{ij} = 0$ when no edge exists. We exclude self-loops, so $w_{ii} = 0$.

A cut. Divide V into two groups A and $V \setminus A$. An edge is *cut* if its endpoints are in different groups. Its weight contributes to our score. The Max-Cut problem is

$$\max_{A \subseteq V} F(A), \qquad F(A) = \sum_{\substack{i<j \\ \mathbf{1}[i\in A]\neq \mathbf{1}[j\in A]}} w_{ij}. \tag{2}$$

The indicator $\mathbf{1}[P]$ equals 1 when proposition P is true, otherwise 0.

For the four-cycle in Figure 1, choosing $A = \{1, 3\}$ cuts all four edges. Choosing $A = \{1, 2\}$ cuts only the edges of weight 2 and 4, scoring 6. *The objective measures crossing-edge weights, not geometric distances between graph vertices.*

Signed weights. Some benchmark graphs have weights $+1$ or -1. Cutting a negative edge *reduces* the objective: we prefer, when possible, to leave its endpoints in the same group. The optimization formula remains valid, but some classical approximation guarantees require *nonnegative* weights.

Why is this hard? There are 2^n sign assignments, but swapping the two groups does not change a cut. Thus there are 2^{n-1} distinct bipartitions. For large n exhaustive search is unrealistic. General weighted Max-Cut is NP-hard: no known polynomial-time algorithm solves all instances exactly [8, 1].

2 Replace sets by signs

Assign every vertex a variable

$$s_i = \begin{cases} +1, & i \in A, \\ -1, & i \notin A. \end{cases} \qquad s \in \{-1, +1\}^n. \tag{3}$$

The key algebraic trick is

$$\mathbf{1}[s_i \neq s_j] = \frac{1 - s_i s_j}{2}. \tag{4}$$

Check both cases: equal signs yield $1 - 1 = 0$; opposite signs yield $1 - (-1) = 2$. Therefore

$$\boxed{\max_{s\in\{-1,+1\}^n} \frac{1}{2} \sum_{i<j} w_{ij}(1 - s_i s_j).} \tag{5}$$

This is an *integer quadratic optimization* problem. It is quadratic because pairs $s_i s_j$ appear; it is discrete because every s_i has only two possible values. If $C = \frac{1}{2}\sum_{i<j} w_{ij}$, then

$$F(s) = C - \tfrac{1}{2} \sum_{i<j} w_{ij} s_i s_j. \tag{6}$$

Thus maximizing the cut is *equivalent* to

$$\min_{s\in\{-1,+1\}^n} \sum_{i<j} w_{ij} s_i s_j.$$

Changing the objective to minimization does not make the discrete problem convex. The analogous continuous minimization is

$$\min_{\|y_i\|=1\ \forall i} \sum_{i<j} w_{ij} y_i^\top y_j.$$

Tiny numerical example. Take three vertices and edges $w_{12} = 2$, $w_{23} = 1$, $w_{13} = -1$. If $s = (1, -1, 1)^\top$, edges 12 and 23 cross but edge 13 does not. Hence $F(s) = 2 + 1 = 3$. By Eq. (5),

$$\tfrac{1}{2}[2(1 - (-1)) + 1(1 - (-1)) - 1(1 - 1)] = 3.$$

We have already turned a combinatorial question into algebra.

3 Why do matrices enter the story?

Adjacency matrix. Place all edge weights into the symmetric matrix $W = (w_{ij}) \in \mathbb{R}^{n\times n}$. For our three-vertex example,

$$W = \begin{pmatrix} 0 & 2 & -1 \\ 2 & 0 & 1 \\ -1 & 1 & 0 \end{pmatrix}. \tag{7}$$

Every row describes one vertex's weighted neighbors. Most large graphs are *sparse*: each vertex is connected to relatively few others, so the project stores W in compressed-sparse-row (CSR) format.

Graph Laplacian. Let $D = \mathrm{diag}(W\mathbf{1})$, with $\mathbf{1}$ the all-ones vector, and define

$$L = D - W. \tag{8}$$

Then an exact identity is

$$s^\top L s = \sum_{i<j} w_{ij}(s_i - s_j)^2. \tag{9}$$

For signs, $(s_i - s_j)^2$ is 4 if cut and 0 otherwise, so

$$\boxed{F(s) = \tfrac{1}{4} s^\top L s.} \tag{10}$$

When all weights are nonnegative, L is positive semidefinite. With *negative* weights this need not be true; our later SDP formulation remains meaningful nevertheless.

A matrix of pairwise relationships. Introduce

$$X = ss^\top, \qquad X_{ij} = s_i s_j. \tag{11}$$

For the example $s = (1, -1, 1)^\top$,

$$X = \begin{pmatrix} 1 & -1 & 1 \\ -1 & 1 & -1 \\ 1 & -1 & 1 \end{pmatrix}. \tag{12}$$

Its diagonal is all ones; an off-diagonal entry is $+1$ for vertices in the same group and -1 for vertices in opposite groups.

PSD: positive semidefinite. We say a symmetric matrix X is *positive semidefinite* (PSD), written $X \succeq 0$, if

$$z^\top X z \geq 0 \quad \text{for every } z \in \mathbb{R}^n. \tag{13}$$

For $X = ss^\top$, this follows immediately:

$$z^\top X z = z^\top s s^\top z = (s^\top z)^2 \geq 0. \tag{14}$$

All eigenvalues of a PSD matrix are nonnegative. The matrix $ss^\top$ is PSD, has $X_{ii} = 1$, and has *rank one* (all columns are multiples of s).

4 Relaxing the impossible search

A *relaxation* enlarges the feasible set: it drops a difficult restriction to obtain a problem easier to solve. Its best objective can be larger than the true Max-Cut optimum. Such a value is an *upper bound* only if the relaxed problem is solved exactly or a valid numerical certificate is available.

In Eq. (11) the condition $X = ss^\top$ forces rank one and discrete off-diagonal entries. We *drop* that condition, retaining the necessary properties $X \succeq 0$ and $\mathrm{diag}(X) = \mathbf{1}$. This produces

$$\boxed{\begin{aligned} \text{maximize}_{X\in\mathbb{S}^n} \quad & \frac{1}{2}\sum_{i<j} w_{ij}(1 - X_{ij}) \\ \text{subject to} \quad & X \succeq 0, \\ & X_{ii} = 1 \quad (i = 1, \ldots, n). \end{aligned}} \tag{15}$$

Here $\mathbb{S}^n$ is the space of real symmetric $n \times n$ matrices. Using Eq. (8), the same objective is $\frac{1}{4}\mathrm{Tr}(LX)$, where Tr denotes the trace (sum of diagonal entries).

4.1 What is an SDP?

The abbreviation is *semidefinite program* (SDP), *not* "SPD." An SDP optimizes a linear function of a matrix under linear constraints and a PSD constraint. In Eq. (15), every feasible matrix represents a relaxed configuration of all vertex pairs.

Why is it convex? A set is convex when every line segment between two of its points stays inside the set. If $X_1, X_2 \succeq 0$ and $0 \leq t \leq 1$, then for every z,

$$z^\top[tX_1 + (1-t)X_2]z = tz^\top X_1 z + (1-t)z^\top X_2 z \geq 0. \tag{16}$$

Thus the PSD cone is convex. Also, $\mathrm{diag}(tX_1 + (1-t)X_2) = \mathbf{1}$ if both diagonals are $\mathbf{1}$. Finally, the objective is *linear in* X, hence simultaneously convex and concave. Therefore, maximizing it over this convex feasible set is a convex optimization problem. It has no spurious local maxima in the *matrix variable* X.

What is *not* convex? The original feasible set of signs is discrete; the rank-one condition is nonconvex. We discarded that restriction, so the SDP may produce a matrix with fractional entries such as $X_{12} = 0.3$. That matrix is valid for the relaxation but is *not yet a cut*.

5 From an SDP matrix to unit vectors

Every PSD matrix has a Gram factorization

$$X = YY^\top, \quad Y = \begin{pmatrix} y_1^\top \\ \vdots \\ y_n^\top \end{pmatrix} \in \mathbb{R}^{n\times p}, \tag{17}$$

with $p = \mathrm{rank}(X)$ (or more columns if desired). Its entries are inner products:

$$X_{ij} = y_i^\top y_j, \qquad X_{ii} = \|y_i\|^2 = 1. \tag{18}$$

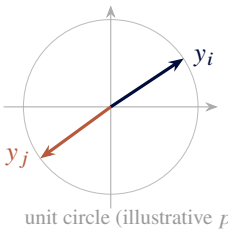


Figure 2. The SDP replaces signs by unit vectors. Opposite vectors have inner product near -1. Later we must map them back to discrete groups.

Thus every vertex is a *unit vector*, a point on the unit sphere in $\mathbb{R}^p$. The inner product has an angular interpretation:

$$y_i^\top y_j = \cos\theta_{ij} \quad \text{when} \quad \|y_i\| = \|y_j\| = 1. \tag{19}$$

If the two vectors point in nearly opposite directions ($\cos\theta \approx -1$), the relaxation favors cutting that edge when $w_{ij} > 0$. For a negative edge, similar directions are often preferable.

Why this helps. Moving vectors continuously on a sphere provides many possible intermediate configurations, unlike the original two-valued signs. But until we perform *rounding*, the vectors do not specify the final partition.

6 The vector optimization and EVA

Substituting Eq. (18) into the SDP objective gives the *vector relaxation*:

$$\boxed{\begin{aligned} \max_{y_1,\dots,y_n \in \mathbb{R}^p} \quad & \Phi(Y) = \frac{1}{2}\sum_{i<j} w_{ij}(1 - y_i^\top y_j) \\ \text{subject to} \quad & \|y_i\| = 1 \ \forall i. \end{aligned}} \tag{20}$$

If p is unrestricted, it represents the full SDP Gram geometry. In our scalable experiments, $p \ll n$, reducing storage dramatically. But Eq. (20) is *nonconvex in* Y: the equality constraints $\|y_i\| = 1$ define spheres, and the objective contains products of vector coordinates.

The distinction in one sentence. The *full SDP* optimizes a large convex matrix X; *EVA* optimizes a smaller, nonconvex factor Y. Both use the same continuous Max-Cut objective after applying $X = YY^\top$, but they solve different numerical formulations [4].

6.1 EVA means Efficient Vector Ascent

EVA is a greedy *block coordinate-ascent* algorithm: it optimizes one vertex vector at a time while holding all other vectors fixed. We can derive its update with just algebra.

For vertex i, define its weighted neighbor field

$$h_i = \sum_{j:\{i,j\}\in E} w_{ij} y_j. \tag{21}$$

Of the full objective, the terms that depend on y_i are

$$\Phi_i(y_i) = \text{constant} - \tfrac{1}{2} y_i^\top h_i. \tag{22}$$

Hence maximizing Φ_i is equivalent to

$$\min_{\|y_i\|=1} y_i^\top h_i. \tag{23}$$

By Cauchy–Schwarz, $y_i^\top h_i \geq -\|h_i\|$; equality is attained by pointing y_i directly *against* h_i. Therefore

$$\boxed{y_i \leftarrow -\frac{h_i}{\|h_i\|} \quad (h_i \neq 0).} \tag{24}$$

If $h_i = 0$, this vertex has no directional preference at that moment; our implementation draws a fresh random unit vector. This derivation is the heart of EVA.

6.2 One complete sweep, step by step

1. Choose an embedding dimension p. In our EVA implementation, independently sample all p coordinates of each initial vector from $U(0,1)$, then normalize each row to have Euclidean norm one. (The later rounding directions use Gaussian sampling.)
2. Visit vertices in order $i = 1, 2, \dots, n$.
3. Compute h_i from *current* neighbor vectors. Updates to earlier vertices in this sweep are already reflected.
4. Replace y_i using Eq. (24); continue to the next vertex.
5. Measure the largest angular change, compute or audit the objective, and decide whether to stop.
6. If not converged, begin another sweep from vertex 1.

The update order matters computationally: a *sequential* sweep is not the same as simultaneously replacing every vector using its old neighbors. Our accelerated Numba implementation compiles this loop while retaining the sequential method.

Why does one update help? Let y_i^{old} and y_i^{new} be unit vectors, and θ_i the angle between them. The exact local improvement is

$$\begin{aligned} \Delta\Phi_i &= \tfrac{1}{2}\left[(y_i^{\text{old}})^\top h_i - (y_i^{\text{new}})^\top h_i\right] \\ &= \tfrac{1}{2}\|h_i\|\,(1 - \cos\theta_i) \geq 0. \end{aligned} \tag{25}$$

So an exact nonzero-field update never decreases the continuous objective. We may accumulate these gains incrementally instead of rescanning all edges on every sweep. A numerical implementation should also check this against independently recomputed objectives periodically.

6.3 When does EVA stop?

Let $\theta_i^{(t)}$ denote the angular movement of vertex i during sweep t. We stop when

$$\boxed{\max_i \theta_i^{(t)} < \varepsilon, \qquad \theta_i^{(t)} = \arccos\left((y_i^{\text{old}})^\top y_i^{\text{new}}\right).} \tag{26}$$

The argument to arccos is clipped to $[-1, 1]$ to handle floating-point roundoff. The tolerance ε is an *angle in radians*, not a bound on the objective error or the distance to the globally optimal cut. We also specify a maximum number of sweeps. Reaching that cap is a different status from numerical convergence.

How precise should ε be? The original experiment used 10^{-3}; the high-precision follow-up tests 10^{-6}. Lower tolerance means a stricter stopping test, potentially many more sweeps, not necessarily a better discrete cut. Near $\theta = 0$, $\arccos(1-\delta)$ is sensitive to floating-point rounding. An extreme target such as 10^{-12} is generally not reliable under a naive double-precision angle test.

6.4 Rank, efficiency, and guarantees

A common default in the EVA study and our repository is

$$p = \left\lceil \sqrt{2(n+1)} \right\rceil. \tag{27}$$

For $n = 800$, $p = 41$; for $n = 20{,}000$, $p = 201$. The factor Y needs np floating-point numbers; the dense X needs n^2. At $n = 20{,}000$ in float64, the raw factor is about 32.2 MB while a single dense Gram matrix occupies about 3.2 GB (decimal units). A complete general-purpose SDP solver may need much more memory than one Gram matrix.

EVA performs a sparse weighted neighbor aggregation for each vertex, giving approximately $O(mp)$ arithmetic per sweep, where $m = |E|$, and $O(m + np)$ sparse-graph-plus-factor memory. Low-rank Burer–Monteiro theory gives useful relationships between rank, stationarity, and SDP optima [4, 5]; however, a particular stopped, nonconvex numerical run is *not* a certified globally optimal SDP solve. Likewise, a high continuous value is not proof of an optimal discrete cut.

7 What exactly is rounding?

A continuous vector solution Y may contain real numbers, but Max-Cut requires exactly two groups. *Rounding* is any rule that converts the row vectors $y_1, \ldots, y_n$ into a label $c_i \in \{0, 1\}$ or sign $s_i \in \{-1, 1\}$ for every vertex.

The actual cut value of the resulting partition is evaluated on the *original graph*:

$$F(c) = \sum_{i<j} w_{ij} \mathbf{1}[c_i \neq c_j]. \tag{28}$$

This is not the same as the continuous objective $\Phi(Y)$. Because rounding loses information, two methods applied to the *same* Y can find different cuts.

A stochastic algorithm. An algorithm is *randomized* when it samples internal random values. One execution is a *trial* or *restart*. Repeating it with independent seeded random choices may produce a different partition. The core question of this work is whether clustering the vectors can outperform a single random separating hyperplane [2, 3].

8 Method 1: the original random hyperplane

Goemans–Williamson (GW). This is the classic randomized rounding method for Max-Cut [1]. It is called `FJ_EVA` in our saved EVA experiments for consistency with a more general rounding interface; *for $k = 2$ it is GW*, not a new random algorithm.

1. Start with the *fixed*, saved matrix $Y \in \mathbb{R}^{n \times p}$.

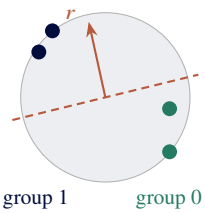


Figure 3. The random direction r defines a perpendicular separating line in $p = 2$; in higher dimensions it defines a hyperplane. Each side becomes a cut group.

2. Draw a random direction $r \in \mathbb{R}^p$ by sampling each coordinate from a standard normal distribution $\mathcal{N}(0, 1)$.
3. For each vertex compute the scalar projection $q_i = y_i^\top r$.
4. Assign $c_i = 1$ when $q_i \geq 0$, and $c_i = 0$ otherwise.
5. Compute $F(c)$ from Eq. (28); save the partition and score.

Equivalently,

$$c_i = \mathbf{1}[y_i^\top r \geq 0]. \tag{29}$$

For example, with $p = 2$, $y_1 = (1, 0)$, $y_2 = (-1, 0)$, and random direction $r = (2, 1)$, we obtain $q_1 = 2$ and $q_2 = -2$, hence different groups. If their edge weight is 3, this edge contributes 3 to the resulting cut. Another draw of r can change assignments of other vertices.

The hyperplane perpendicular to r and passing through the origin separates the sphere into two hemispheres. Two vectors with angle θ_{ij} are separated with probability

$$\Pr(c_i \neq c_j) = \frac{\theta_{ij}}{\pi}. \tag{30}$$

For graphs with nonnegative weights and a suitably solved SDP, GW's celebrated expected approximation ratio is at least about 0.87856 of the true Max-Cut optimum [1]. *This statement must not be transferred without qualification to signed-weight G-set instances or arbitrary nonconvex EVA factors.*

9 Method 2: K-Means2N

Clustering. A *cluster* is a collection of similar vectors. In k-means, each cluster is represented by a center (*centroid*), and each point is repeatedly assigned to the nearest center. Here $k = 2$ is the *number of clusters*, not the embedding dimension p.

Because our vectors all have length one, we use *spherical k-means*. Each centroid is normalized after updating. For unit vectors u, v,

$$\|u - v\|^2 = 2(1 - u^\top v), \tag{31}$$

so maximizing cosine similarity $u^\top v$ is equivalent to minimizing Euclidean distance between normalized vectors [2].

9.1 Algorithm, step by step

1. Sample two *independent* Gaussian vectors $r_1, r_2 \in \mathbb{R}^p$.
2. Normalize them: $\mu_j = r_j / \|r_j\|$ for $j = 1, 2$.
3. Assign each vertex to the center having the larger inner product:

$$c_i \leftarrow \arg\max_{j \in \{1,2\}} y_i^\top \mu_j. \tag{32}$$

4. For each cluster, sum its member vectors and normalize:

$$\mu_j \leftarrow \frac{\sum_{i:c_i=j} y_i}{\left\|\sum_{i:c_i=j} y_i\right\|}. \tag{33}$$

5. Repeat assignment and centroid updates until labels stop changing or 100 iterations have been attempted.
6. Return the resulting labels; score the partition on the original weighted graph.

In the implementation, empty or zero-sum clusters receive a randomly chosen data-vector center. The *two independent initial directions* distinguish K-Means2N from K-MeansNM.

What k-means optimizes. It makes vectors compact around their centroids (equivalently, increases total within-cluster cosine similarity). It does *not* directly optimize the graph cut $F(c)$. An improvement in clustering compactness may or may not improve the cut. K-Means2N has no universal guarantee of outperforming a paired GW cut [2].

10 Method 3: K-MeansNM

K-MeansNM is also spherical 2-means, but its *initialization and selection* are different. The method uses an *antipodal initialization*: the second initial direction is the negative of the first.

1. Draw one Gaussian direction $r \in \mathbb{R}^p$ and normalize it.
2. Initialize opposite centroids:

$$\mu_1 = \frac{r}{\|r\|}, \qquad \mu_2 = -\mu_1. \tag{34}$$

3. Assign each vertex by Eq. (32). Because the centers are opposite, this is exactly the same initial bipartition as a GW hyperplane with direction r (up to label names and zero ties).
4. Calculate the actual weighted cut $F(c)$ and store the best labels seen so far.
5. Perform the spherical centroid updates and reassignments as in K-Means2N. After *every assignment*, recompute $F(c)$ and retain it if it is better than the saved best.
6. Stop at stable labels or the iteration limit; return the *best encountered cut*, possibly from an intermediate iteration rather than the final clustering assignment.

Why retaining the initial cut matters. The first assignment is a valid GW cut. Since our implementation saves the maximum $F(c)$ over the initial and subsequent assignments,

$$F_{\mathrm{NM,kept}} \geq F_{\mathrm{GW,paired}}. \tag{35}$$

This is a *per-trial implementation guarantee* for the same Gaussian direction r; it is not a proof that k-means clustering monotonically increases the cut at every iteration. Clustering can temporarily worsen it, and Eq. (35) does not establish a better worst-case approximation ratio for arbitrary signed-weight graphs.

A controlled comparison. In the repository, GW and K-MeansNM use matched initial random directions for the same graph and restart index. Thus one can compare the two starting from a common hyperplane. K-Means2N uses two independent random directions and a different random stream. All three are still evaluated using the exact same graph and continuous factor Y.

11 Why not round just once?

Consider one expensive continuous optimization producing Y. The graph and the vector factor now remain *fixed*. Randomized hyperplanes and randomized centroid initializations may yield different partitions on different trials. Therefore we can run the *cheap rounding stage many times* without solving another SDP or running EVA again.

What a restart means here. One rounding restart is one new draw of the random direction(s), followed by the relevant clustering updates and one saved partition. It is *not* a new EVA run, a new graph, or a new SDP solve. With R restarts per method and three methods, we obtain $3R$ candidate cuts per graph from the same Y.

$$\begin{aligned} R = 20: &\quad 60 \text{ cuts per graph,} \\ R = 100: &\quad 300 \text{ cuts per graph,} \\ R = 1{,}000: &\quad 3{,}000 \text{ cuts per graph.} \end{aligned} \tag{36}$$

For 71 graphs, 1,000 trials of each of three methods would yield 213,000 candidate cuts *if every graph has a completed factor*. These trials improve our *search among discrete cuts*; they do not change the fixed continuous objective $\Phi(Y)$.

Why keep all scores? Reporting only the largest cut can hide variability. For scores $F_1, \ldots, F_R$, report at least

$$\bar{F} = \frac{1}{R}\sum_{r=1}^{R} F_r, \quad F_{\max} = \max_{1\leq r\leq R} F_r, \tag{37}$$

and preferably their median, standard deviation, trial count, seed, and factor identity. A method may have a high best cut but unstable typical performance.

A probability illustration. If (hypothetically) each independent trial has probability q of finding a specified quality threshold, the chance of finding it at least once in R independent trials is

$$1 - (1 - q)^R. \tag{38}$$

This is why more restarts can help. But restarts from the *same factor* are not independent graph instances. They may share biases and cannot fix a poor continuous factor automatically.

12 What are the G-set graphs?

The *G-set* is a widely used benchmark collection of weighted Max-Cut graphs created using Rinaldi's graph generator and distributed through Y. Ye's resource. It includes toroidal, nearly

planar, and random graph topologies. The canonical collection used in this project has 71 instances:

$$\{\mathrm{G1}, \ldots, \mathrm{G67}, \mathrm{G70}, \mathrm{G72}, \mathrm{G77}, \mathrm{G81}\}. \tag{39}$$

They contain approximately 800 to 20,000 vertices with edge weights ±1 [2, 3].

Why is G-set famous? Researchers need common test problems to compare algorithms. If several papers use the same named graph and scoring convention, one can compare the best attained cut, convergence, memory, and run time. Many instances have no proven discrete optimum, so a published *best-known cut* is not automatically a mathematical optimum.

Historical context. The 2020 clustering paper and 2021 dissertation report that their full-SDP tools could not solve several large instances, including G55, G56, G60, G61, G70, G77, and G81 [2, 3]. That is an observation about their solver/resources, *not* a proof these graphs cannot be processed by other algorithms. EVA's factor representation makes such larger instances much more practical.

Why exact input identity matters. Two files with the same graph name may differ in format, edge ordering, or weights. Our repository uses the canonical `data/G*.txt` files and records a SHA256 hash of each input. A valid comparison must refer to the same *weighted* graph and objective convention.

13 SDP once, rounding many times

The essential pipeline. The large optimization problem is solved once for a *fixed graph, formulation, seed, rank and tolerance*; then the saved continuous vectors are reused for independently randomized rounding trials:

$$\underbrace{G \longrightarrow Y}_{\substack{\text{EVA once} \\ \text{for fixed settings}}} \longrightarrow \underbrace{c^{(1)}, \ldots, c^{(R)}}_{\text{round many times}} \longrightarrow F(c^{(r)}). \tag{40}$$

The conventional convex-SDP route is $G \to X \succeq 0 \to Y \to$ rounding. The EVA route is $G \to Y \to$ rounding; *EVA never needs to form the dense X explicitly*. If we change the graph, EVA rank, optimization seed, or convergence tolerance, we have a *new optimization experiment* and should not silently reuse an incompatible checkpoint.

What is saved in the project? The project's EVA command writes a per-graph factor archive `factors/Gxx.npz` containing all vectors Y, convergence metadata and RNG state; `logs/Gxx-sweeps.jsonl` for the optimization trajectory; `cases/Gxx.json` for summary and provenance; and a verified partition archive with each method's individual labels and scores. A separate `eva_rounding_cli` can append nonoverlapping restart-index ranges using the *same verified factor*, while keeping exact snapshots and hashes.

Why hashes and checkpoints? They make it possible to verify that a later rounding batch really used the intended graph and vectors and that the optimization can resume after interruption. One should never mix scores from different factor versions as if they came from the same continuous solution.

Two meanings of "solve". A completed *EVA solve* means the continuous factor optimizer converged according to Eq. (26) or reached its cap. A completed *rounding trial* means it produced a valid binary partition and an exact weighted cut value. Neither event alone establishes the globally optimal discrete Max-Cut value.

14 Optional: local search after rounding

The repository also supports *one-vertex-flip local search* as an additional postprocessing step. Starting from any rounded sign vector s, flip one sign $s_i \leftarrow -s_i$ if doing so improves the cut. The exact gain is

$$\Delta_i F = s_i \sum_j w_{ij} s_j. \tag{41}$$

To derive it, only edges adjacent to i change their contribution. Accepting only positive gains guarantees that each accepted flip increases the actual cut. Repeat until no improving flip is found or the sweep budget is exhausted. The saved methods `FJ-Flip_EVA` and `K-MeansNM-Flip_EVA` apply this to their corresponding initial rounding results. This is a *local heuristic*, not an exact Max-Cut solver.

15 What changes for Max-k-Cut?

Everything above has concentrated on $k = 2$: two groups, one sign per vertex. For Max-k-Cut, we partition vertices into k groups ($k \geq 2$), and the discrete objective is still the weight of edges whose endpoints have different labels.

An important SDP relaxation for Max-k-Cut is the Frieze–Jerrum program

$$\begin{aligned} \max_{X \succeq 0} \quad & \frac{k-1}{2k} \operatorname{Tr}(LX) \\ \text{subject to} \quad & X_{ii} = 1 \ \forall i, \\ & X_{ij} \geq -\frac{1}{k-1} \ \forall i \neq j. \end{aligned} \tag{42}$$

At $k = 2$, the lower bound is -1, which follows automatically from PSD and unit diagonal; Eq. (15) is recovered. For $k > 2$, that additional constraint is generally essential. One can use k Gaussian directions for a corresponding randomized rounding step, but the binary EVA implementation and the three named two-cluster methods in this tutorial *do not by themselves solve the general Max-k-Cut SDP*. See [6, 7] for broader perspectives.

16 How should we judge an experiment?

1. **Verify the input.** Record graph name, vertex and edge counts, weights and SHA256.
2. **Describe the optimizer.** Say whether Y came from Numba EVA, NumPy EVA, or a factorization of convex-SDP output; report rank, tolerance, seed, sweeps and timing.
3. **Separate objectives.** Report the continuous $\Phi(Y)$ and the actual integer $F(c)$ separately. Do not call the first an attained discrete cut.

4. **Specify all rounding budgets.** Each method should have named seeds, restart indices, number of trials and a reproducible partition archive.
5. **Compare fairly.** Use the same saved factor when isolating the effect of GW versus K-Means2N versus K-MeansNM. Changing the optimizer and rounder simultaneously confounds the comparison.
6. **Measure more than the best.** Report best, mean, median, dispersion and paired differences when methods share the initial hyperplane.
7. **Be careful with guarantees.** For signed graphs, do not assume the nonnegative-weight GW 0.87856 theorem applies. Numerically approximate SDP primal objectives are not automatically certified bounds.

17 A complete mental picture

We began with a graph whose vertices must be separated into two groups. Signs made the cut objective algebraic. A rank-one matrix $ss^\top$ encoded those signs, but the rank restriction was difficult. Dropping it produced a convex SDP over PSD matrices. Such matrices can be represented as inner products of unit vectors. EVA works directly with a low-rank collection of those vectors and repeatedly optimizes one vector at a time, using a simple closed-form update. Finally, randomized rounding or spherical clustering converts the continuous vectors back to actual binary graph partitions.

Graph → signs → Gram matrix →
SDP/vector relaxation → EVA factor Y →
GW / K-Means2N / K-MeansNM → cut.

The key experimental economy is that *the expensive continuous solution can be reused*. Thousands of rounding trials explore many cuts with a fixed factor, while changes in EVA tolerance or rank require a separately identified continuous experiment.

Glossary

Cut
A partition of graph vertices into two sets.

Objective
The number or expression we maximize or minimize.

Relaxation
A larger feasible set that simplifies optimization.

PSD
A symmetric matrix with $z^\top Xz \geq 0$ for every z.

SDP
A semidefinite program: linear matrix optimization with PSD constraints.

Gram matrix
A matrix of pairwise inner products $X_{ij} = y_i^\top y_j$.

Embedding dimension
The number p of coordinates per vector.

EVA sweep
One sequential visit through all vertex vectors.

Rounding
Turning continuous vectors into discrete labels.

Centroid
A representative direction of a cluster.

Restart
A new random rounding initialization on a fixed factor.

Convergence tolerance
A threshold on numerical change; not a global optimality guarantee.

Sources and further reading

References

[1] M. X. Goemans and D. P. Williamson (1995). Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *J. ACM* 42(6), 1115–1145. https://doi.org/10.1145/227683.227684.

[2] A. E. Rodríguez-Fernández, B. González-Torres, R. Menchaca-Méndez, and P. F. Stadler (2020). Clustering improves the Goemans–Williamson approximation for the Max-Cut problem. *Computation* 8(3), 75. https://doi.org/10.3390/computation8030075.

[3] A. E. Rodríguez Fernández (2021). *On the Study of Fitness Landscapes and the Max-Cut Problem*. Doctoral dissertation, Leipzig University, especially Chapter 6. (User-provided thesis.)

[4] B. González-Torres, A. E. Rodríguez-Fernández, R. Menchaca-Méndez, R. Menchaca-Méndez, and O. Schütze (2026). EVA: A fast greedy optimization algorithm for the Max-Cut vector relaxation problem. *Results in Control and Optimization* 24, 100759. https://doi.org/10.1016/j.rico.2026.100759. Original MATLAB code: https://github.com/b3r8/EVA.

[5] S. Burer and R. D. C. Monteiro (2003). A nonlinear programming algorithm for solving semidefinite programs via low-rank factorization. *Mathematical Programming* 95, 329–357. https://doi.org/10.1007/s10107-002-0352-8.

[6] A. Frieze and M. Jerrum (1997). Improved approximation algorithms for MAX k-CUT and MAX BISECTION. *Algorithmica* 18, 67–81. https://doi.org/10.1007/BF02523688.

[7] P. Felzenszwalb, C. Klivans, and A. Paul (2022). Clustering with semidefinite programming and fixed point iteration. *J. Mach. Learn. Res.* 23, paper 21-0402. https://jmlr.org/papers/v23/21-0402.html.

[8] R. M. Karp (1972). Reducibility among combinatorial problems. In *Complexity of Computer Computations*, 85–103.